

Easy3E: Feed-Forward 3D Asset Editing via Rectified Voxel Flow

Shimin Hu Yuanyi Wei Fei Zha Yudong Guo* Juyong Zhang

University of Science and Technology of China

<https://ustc3dv.github.io/Easy3E/>



Figure 1. We introduce Easy3E, a novel method for 3D asset editing. Guided by a single edited view and a coarse 3D mask, our method can perform both significant geometric changes and fine-grained appearance edits. Easy3E efficiently produces globally consistent and high-fidelity 3D results, demonstrating its power and flexibility across diverse assets.

Abstract

Existing 3D editing methods rely on computationally intensive scene-by-scene iterative optimization and suffer from multi-view inconsistency. We propose an effective and feed-forward 3D editing framework based on the TRELIS generative backbone, capable of modifying 3D models from a single editing view. Our framework addresses two key issues: adapting training-free 2D editing to structured 3D representations, and overcoming the bottleneck of appearance fidelity in compressed 3D features. To ensure geometric consistency, we introduce Voxel FlowEdit, an edit-driven flow in the sparse voxel latent space that achieves globally consistent 3D deformation in a single pass. To restore high-fidelity details, we develop a normal-guided single to multi-view generation module as an external appearance prior,

successfully recovering high-frequency textures. Experiments demonstrate that our method enables fast, globally consistent, and high-fidelity 3D model editing.

1. Introduction

3D asset editing is a fundamental task in numerous applications, such as gaming, film production, architectural visualization, and emerging fields like AR/VR and digital twins. Therefore, developing intuitive and efficient editing tools has been a long-standing challenge in computer graphics and 3D computer vision. The primary goal is to enable users to perform complex 3D modifications through simple and easy-to-use inputs, such as 2D conditions or text prompts. A framework that can translate these user inputs into coherent and high-fidelity 3D results will significantly simplify the content creation process and make 3D editing

*Corresponding Author.

easily accessible to more users.

Existing 3D editing methods span multiple paradigms. Classical approaches follow the 2D-lifting pipeline [12, 44, 59], where edited 2D images supervise the optimization of a 3D representation (e.g., NeRF [31] or 3DGS [18]). Although effective for appearance-level edits, these optimization-based methods require per-scene iterative refinement and depend heavily on multi-view coverage, making them fragile when the edit introduces noticeable geometric deviation from the original asset. More recent works adopt multi-view or view-consistent diffusion models [2], which improve cross-view consistency but still operate in a 2D-native feature space, requiring the model to infer 3D structure implicitly during generation. Such implicit reasoning limits their ability to handle edits that alter shape, topology, or volumetric occupancy, as 2D features alone provide insufficient cues for reliable 3D structural inference. Both categories rely on image-space features and therefore struggle with large geometric changes and precise structural control, especially when edits demand explicit, globally consistent manipulations of the underlying 3D structure.

In contrast, 3D-native generative models [15, 42, 48] learn explicit, structured 3D latent fields directly from data. These representations encode geometry natively rather than reconstructing it from multiple images, opening a fundamentally different editing perspective: instead of optimizing a 3D scene or manipulating multi-view features, one can directly modify the underlying structured 3D latent space where geometry is explicitly parameterized. This paradigm promises feed-forward, globally coherent 3D editing, but introduces two key challenges.

First, the lack of paired 3D editing data necessitates adapting training-free 2D editing techniques [13, 30, 32] to 3D latent fields. However, many 2D methods depend on architectural components that do not transfer to 3D, such as manipulations of cross-attention maps [4, 13] or 2D-specific feature maps [43]. Second, structured 3D generative models [15, 48] utilize compact latent tokens to ensure geometric consistency and fast inference. This compression limits their ability to represent high-frequency texture details, leading to oversmoothed or low-fidelity appearance. Thus, the core technical questions are: (1) how to redesign training-free 2D editing approaches to operate on structured 3D latent spaces, and (2) how to restore high-quality texture details on edited geometry given limited 3D appearance priors.

To address these challenges, we propose a fully feed-forward 3D editing framework built on the TRELLIS generative backbone [48]. Given a single edited view and a user-defined editable region, our method performs both geometric and appearance editing directly in TRELLIS’s sparse voxel latent space. We introduce Voxel FlowEdit, a latent-

space editing mechanism that translates source voxels to target voxels via an adapted velocity field, enabling globally coherent geometric deformation in a single pass. Following this coarse transformation, we apply a structured latent repainting stage to locally refine geometry and appearance while anchoring unedited regions, ensuring consistency and detail preservation.

To overcome the limited appearance priors of 3D generative models, we further incorporate an optional normal-guided multi-view generative module. It synthesizes high-fidelity auxiliary views aligned with the edited geometry, providing rich 2D appearance cues that enhance texture realism in the final 3D asset.

In summary, our main contributions are as follows:

- We construct an effective and feed-forward framework that leverages the powerful prior of 3D generative models to enable efficient and high-quality 3D asset editing from a single edited view.
- We introduce Voxel FlowEdit, a voxel-flow editing mechanism. It constructs the source-to-target translation of 3D assets within the sparse voxel latent space by utilizing a specially adapted velocity field, achieving globally coherent 3D geometric deformation.
- We develop a dedicated normal-guided single-to-multi-view generation model which serves as an external appearance prior to overcome the limitation of compressed 3D appearance representations, restoring high-fidelity textures onto the edited geometry.

2. Related Work

3D Model Generation. Recent progress in 3D generative modeling has evolved from lifting 2D observations [24, 36] to learning fully 3D-native representations that jointly model geometry and appearance [15]. Early image-to-3D frameworks employed NeRF or mesh decoders to reconstruct assets from few views [31, 34, 35, 57], while subsequent diffusion-based pipelines leveraged 2D priors for text-to-3D synthesis via score distillation [24, 36, 38, 45]. To improve view consistency and scalability, several works proposed generating multi-view images as intermediate supervision before reconstructing 3D assets [27, 28, 41, 51], achieving better alignment but still constrained by 2D lifting. More recently, large-scale 3D-native frameworks have emerged, learning structured latent spaces directly from massive 3D corpora [15, 23, 42, 48, 50, 58]. These models enable feed-forward generation of meshes, radiance fields, or 3D Gaussian representations conditioned on images or text, substantially advancing fidelity and controllability. Parallely, autoregressive models like SAR3D [7] utilize multi-scale 3D VQVAE for unified generation and understanding, while OctGPT [46] leverages octree-based representations to scale up 3D shape synthesis.

2D Image Editing. Early diffusion-based image editing reconstructs a given image by inverting it into the latent space of a pretrained model and then applying localized manipulations via attention control or latent blending to realize semantic and structural changes [4, 13, 17, 30, 32, 43, 49, 53]. In parallel, training-based approaches adapt the generator or lightweight adapters to the target domain, improving edit fidelity and controllability through finetuning or conditioning modules such as DreamBooth [20, 39], LoRA [16], and ControlNet [11, 55]. In contrast to diffusion-style U-Net editors, inversion-free flow-matching formulations directly construct continuous source-target transformations in the learned velocity field, avoiding iterative inversion and aligning naturally with DiT-style architectures [19, 26, 33]. Given that paired 3D editing data are largely unavailable and that the underlying generative backbone adopts a flow-matching formulation [26, 29], this compatibility makes the flow-based editing paradigm well suited to our setting.

3D Model Editing. Recent 3D editing methods are characterized by how 2D guidance is coupled to the 3D representation. A first line iteratively optimizes 3D representations by supervising rendered views with 2D-edited images, typically via score-distillation losses [12, 40, 59]. Subsequent work improves robustness and efficiency by synchronizing multi-view constraints or imposing geometry-aware priors during optimization [1, 3, 47]. A parallel direction leverages multi-view or video diffusion to produce edited view sets with stronger viewpoint coverage and spatial-temporal regularization before lifting back to 3D, enabling multi-view propagation of edits [6]. More recently, 3D-native generative backbones have begun to explore feed-forward editing. These approaches typically rely on local mechanisms such as masked reconstruction [9] or multi-view inpainting [2] to localize changes. While these methods improve efficiency, their reliance on local masking or inpainting often limits them to textural changes or simple additions, struggling with edits that require globally coherent geometric deformation. Contemporaneous to our work, VoxHammer [21] and Nano3D [52] independently explore a highly similar pipeline for feed-forward editing on 3D-native generative models. Developed in parallel, our study shares the core motivation of enabling direct manipulations within the 3D latent space. These works collectively demonstrate the timeliness and significance of shifting towards fully 3D-native feed-forward editing paradigms.

3. Method

We introduce an effective, feed-forward framework that achieves high-fidelity 3D asset editing. Given a source 3D asset $\mathcal{A}_{\text{src}}$, a 3D region mask $\mathcal{M}$, and a target image I^{tgt} obtained by editing a rendered view I^{src} , our method produces an edited asset with consistent geometry and appearance.

The overall pipeline is illustrated in Fig. 2. We first establish our model foundation by formalizing the structured latent representation (Sec. 3.1). For precise geometric editing in the 3D sparse voxel domain, we introduce a novel Flow Matching-based voxel editing algorithm (Sec. 3.2). For generating new latent features that define the edited geometry and appearance while maintaining source consistency, we propose a SLAT repainting technique (Sec. 3.3). Finally, for enhancing visual realism, we employ a texture refinement module that improves fidelity using normal-guided multi-view images generation (Sec. 3.4).

3.1. Structured Latent Representation

Our editing method operates directly in the structured 3D latent space used by TRELLIS [48]. Specifically, the Structured LATent (SLAT) representation is defined as

$$\mathbf{Z} = (\mathcal{V}, \{\mathbf{z}_{\mathbf{p}}\}_{\mathbf{p} \in \mathcal{V}}),$$

where the active voxel set $\mathcal{V}$ consists of voxels that intersect the surface of the 3D mesh, and $\mathbf{z}_{\mathbf{p}}$ is a local latent feature attached to each active voxel. The local latent features are obtained by fusing multi-view image features (e.g., DINOv2) projected onto these voxels. TRELLIS uses two rectified flow transformers to respectively predict the voxel structure $\mathcal{V}$ and the local latent field $\{\mathbf{z}_{\mathbf{p}}\}$. The resulting SLAT representation can be further decoded into 3DGS, mesh, or NeRF.

TRELLIS is trained under the rectified flow framework, which learns a deterministic velocity field

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}_{\theta}(\mathbf{x}, t),$$

following a linear path

$$\mathbf{x}(t) = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1,$$

where $\mathbf{x}_0$ is a clean sample from the SLAT data distribution at $t = 0$, and $\mathbf{x}_1$ is its corresponding noise sample at $t = 1$. This noise-to-data formulation provides the flow-based perspective on which we construct edit-driven trajectories in the structured latent space.

3.2. Sparse Voxel Editing

Our editing process begins at the structural level. The voxel structure $\mathcal{V}$ is represented as a binary occupancy grid, which is encoded by a 3D VAE into a low-dimensional continuous latent vector $\mathbf{x}$. Our goal is to propagate 2D edit signals through this latent voxel space and transform the source latent $\mathbf{x}^{\text{src}}$ into a target latent $\mathbf{x}^{\text{tgt}}$ conditioned on the target image I^{tgt} .

Editing Trajectory Modeling. Inspired by FlowEdit [19], we model the structural editing process as a continuous trajectory in the latent space. Let $\mathbf{x}_t$ denote the latent structural

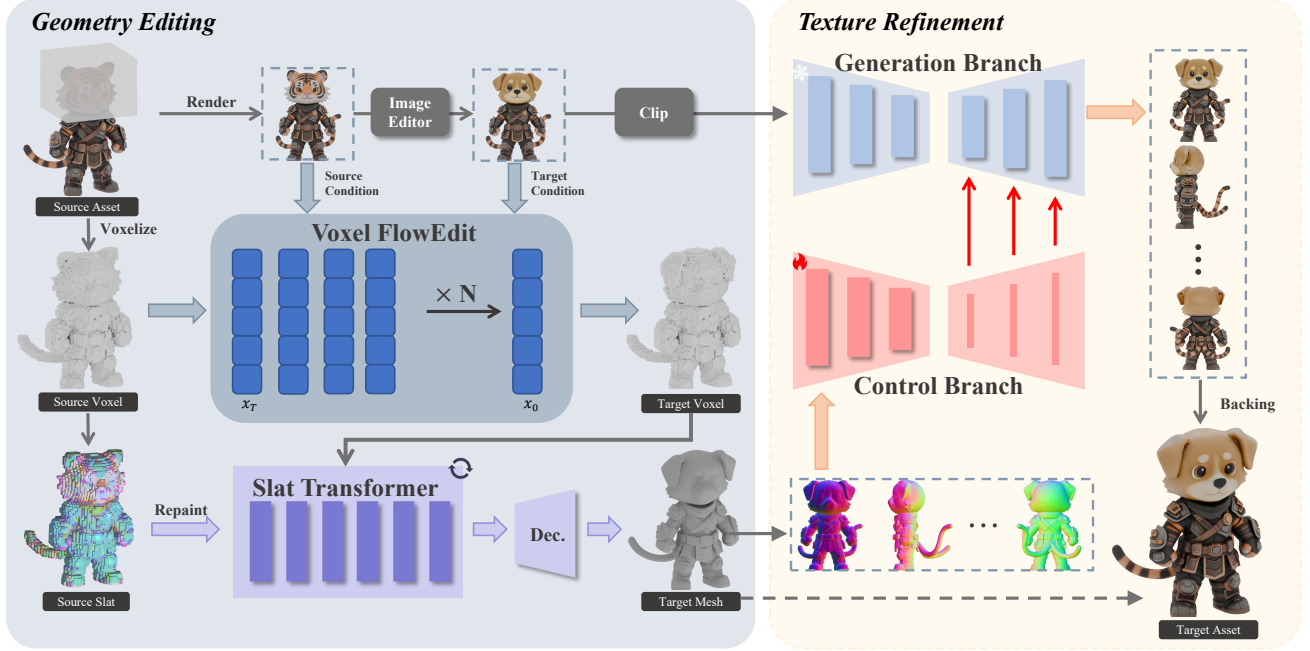


Figure 2. Overview of Easy3E. The framework operates in two main stages: Geometry Editing and Texture Refinement. Starting from a rendered source view, an edited target image provides the guidance for editing. In the Geometry Editing stage, the Voxel FlowEdit algorithm transforms the source voxel structure under flow-based guidance, followed by SLAT Repainting that refines local latent features to produce the target mesh. The Texture Refinement stage then employs a generation branch and a normal-guided control adapter to synthesize multi-view-consistent textures, which are projected and fused onto the mesh to yield the final high-fidelity 3D asset.

state at time $t \in [0, 1]$, and let $\mathbf{v}_{\text{edit}}(\mathbf{x}_t, t)$ be the edit-driven velocity field. The trajectory is defined by the masked ODE

$$d\mathbf{x}_t = \mathcal{M}_\ell \odot \mathbf{v}_{\text{edit}}(\mathbf{x}_t, t) dt, \quad (1)$$

where $\mathcal{M}_\ell$ restricts updates to the editable region. Following the rectified-flow convention, we set

$$\mathbf{x}_{t=1} = \mathbf{x}^{\text{src}}, \quad \mathbf{x}_{t=0} = \mathbf{x}^{\text{tgt}},$$

so that integrating the ODE traces a continuous path from the source latent structure to the desired edited one.

We construct $\mathbf{v}_{\text{edit}}$ by differencing the flow trajectories of the pretrained model under the source and target conditions. For each condition $c \in \{\text{src}, \text{tgt}\}$, the rectified-flow formulation specifies a linear latent path

$$\mathbf{x}_t^c = (1 - t) \mathbf{x}_0^c + t \mathbf{x}_1^c,$$

connecting a clean endpoint $\mathbf{x}_0^c$ and a noisy endpoint $\mathbf{x}_1^c$. The velocity network satisfies

$$\frac{d\mathbf{x}_t^c}{dt} = \mathbf{v}_\theta(\mathbf{x}_t^c, t | I^c).$$

We impose a shared terminal noise state $\mathbf{x}_1^{\text{src}} = \mathbf{x}_1^{\text{tgt}}$ and construct an interpolated path that matches our ODE boundary conditions:

$$\mathbf{x}_t = \mathbf{x}_0^{\text{src}} - \mathbf{x}_t^{\text{src}} + \mathbf{x}_t^{\text{tgt}}.$$

Differentiating this path yields the edit velocity:

$$\mathbf{v}_{\text{edit}}(\mathbf{x}_t, t) = \mathbf{v}_\theta(\mathbf{x}_t^{\text{tgt}}, t | I^{\text{tgt}}) - \mathbf{v}_\theta(\mathbf{x}_t^{\text{src}}, t | I^{\text{src}}). \quad (2)$$

The geometric definition of this vector is visualized in Figure 3(a).

Guided Flow Regularization. The base ODE in Eq. 1 provides an efficient initialization for 3D structural editing. Following FlowEdit [19], the editing velocity $\mathbf{v}_{\text{edit}}$ is estimated by averaging conditional velocity differences over multiple noise samples.

However, this baseline often over-preserved the source structure and fails to fully reach the target edit (Fig. 3(a)), likely due to discretization errors that push the trajectory off the data manifold. To mitigate this drift, we introduce auxiliary guidance terms that refine the evolution while preserving the underlying flow dynamics.

We introduce a silhouette-guidance term based on the energy

$$\begin{aligned} \mathcal{E}_{\text{sil}}(\mathbf{x}) &= \text{BCE}(S(\mathbf{x}), M_{\text{sil}}), \\ S(\mathbf{x}) &= 1 - \exp\left(-\kappa \sum_z p_z(\mathbf{x})\right), \end{aligned} \quad (3)$$

where BCE denotes binary cross-entropy, $p_z(\mathbf{x})$ is the decoded voxel occupancy probability at depth z , and $S(\mathbf{x})$ is

(a) Editing by FlowEdit (b) Editing by Voxel FlowEdit

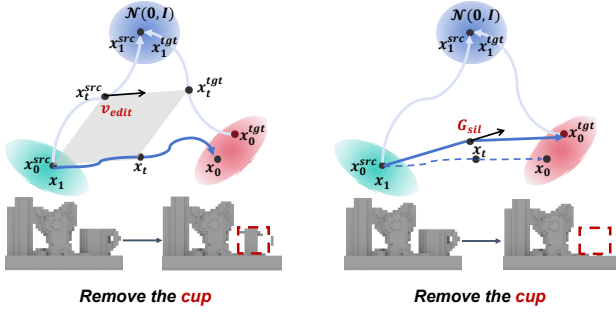


Figure 3. Comparison of FlowEdit’s limitations and the Voxel FlowEdit solution. (a) Base FlowEdit: The semantic velocity $\mathbf{v}_{\text{edit}}$ is corrupted by accumulated approximation error, causing the trajectory to drift and resulting in structural corruption (red dashed box). (b) Voxel FlowEdit: The edit is driven by external gradient guidance $\mathbf{G}_{\text{sil}}$, while internal correction ξ_{traj} maintains manifold consistency. This combined approach achieves a clean and structurally integral edit.

the rendered 2D silhouette obtained by accumulating occupancy along each camera ray. The target silhouette M_{sil} is extracted from the edited target view I^{tgt} , and κ controls its sharpness. The guidance term

$$\mathbf{G}_{\text{sil}}(\mathbf{x}_t) = -\nabla_{\mathbf{x}} \mathcal{E}_{\text{sil}}(\mathbf{x}_t)$$

encourages the evolving structure to match the target silhouette.

Directly applying $\mathbf{G}_{\text{sil}}$ may push the edited state off the smooth flow manifold. To counteract this, we introduce a trajectory-consistency correction $\xi_{\text{traj}}(\mathbf{x}_t)$ [19], which projects the perturbed latent state back onto the interpolating manifold:

$$\xi_{\text{traj}}(\mathbf{x}_t) = \hat{\mathbf{x}}_{0|t}^{\text{tgt}} - \hat{\mathbf{x}}_{0|t}^{\text{src}},$$

where the clean-state estimates are obtained by back-projecting each trajectory,

$$\hat{\mathbf{x}}_{0|t}^c = \mathbf{x}_t^c - t \mathbf{v}_{\theta}(\mathbf{x}_t^c, t | c), \quad c \in \{\text{src}, \text{tgt}\}.$$

Combining the semantic velocity $\mathbf{v}_{\text{edit}}$ with these auxiliary terms yields the controllable flow-regularized update:

$$\begin{aligned} d\mathbf{x}_t &= \mathcal{M}_{\ell} \odot \mathbf{v}_{\text{edit}}(\mathbf{x}_t, t) dt \\ &+ \mathcal{M}_{\ell} \odot \left(\Gamma \xi_{\text{traj}}(\mathbf{x}_t) - \eta \mathbf{G}_{\text{sil}}(\mathbf{x}_t) \right) dt, \end{aligned} \quad (4)$$

where the constants Γ and η control the relative strength of the trajectory correction and gradient guidance. The successful edit and stable trajectory achieved by this flow-regularized update are visualized in Fig. 3(b).

To obtain a stable discrete realization, we integrate the above flow in multiple steps from $t=1$ to 0. Our final algorithm is summarized in Algorithm 1.

Algorithm 1 Sparse Voxel FlowEdit

Input: source latent $\mathbf{x}_0^{\text{src}}, \{t_i\}_{i=0}^T, \mathcal{M}_{\ell}, \Gamma, \eta$
Output: edited latent $\mathbf{x}_0^{\text{tgt}}$
Init: $\mathbf{x}_{t_T} \leftarrow \mathbf{x}_0^{\text{src}}$
for $i = T$ **down to** 1 **do**
 Sample $\epsilon_{t_i} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 $\mathbf{x}_{t_i}^{\text{src}} \leftarrow (1 - t_i) \mathbf{x}_0^{\text{src}} + t_i \epsilon_{t_i}$
 $\mathbf{x}_{t_i}^{\text{tgt}} \leftarrow \mathbf{x}_{t_i}^{\text{src}} + \mathbf{x}_{t_i} - \mathbf{x}_0^{\text{src}}$
 $\tilde{\mathbf{x}}_{t_{i-1}} \leftarrow \mathbf{x}_{t_i} + \Delta t \mathcal{M}_{\ell} \odot \mathbf{v}_{\text{edit}}(\mathbf{x}_{t_i}, t_i)$
 $\hat{\mathbf{x}}_{0|t_i}^i \leftarrow \mathbf{x}_{t_i}^i - t_i \mathbf{v}_{\theta}(\mathbf{x}_{t_i}^i, t_i | I^i), \quad i \in \{\text{src}, \text{tgt}\}$
 $\xi_{\text{traj}}(\mathbf{x}_{t_i}) \leftarrow \hat{\mathbf{x}}_{0|t_i}^{\text{tgt}} - \hat{\mathbf{x}}_{0|t_i}^{\text{src}}$
 $\mathbf{G}_{\text{sil}}^{\text{sil}} \leftarrow \mathbf{G}_{\text{sil}}(\tilde{\mathbf{x}}_{t_{i-1}})$
 $\mathbf{x}_{t_{i-1}} \leftarrow \tilde{\mathbf{x}}_{t_{i-1}} + \Delta t \mathcal{M}_{\ell} \odot (\Gamma \xi_{\text{traj}}(\mathbf{x}_{t_i}) - \eta \mathbf{G}_{\text{sil}}^{\text{sil}})$
end for
Return: $\mathbf{x}_0^{\text{tgt}} \leftarrow \mathbf{x}_{t_0}$

3.3. SLAT Repainting

To further refine local geometry and appearance beyond the sparse structural edits, we introduce a latent-level repainting stage that updates voxel features in the editable region while preserving the source characteristics elsewhere.

Building upon the edited sparse voxels $\mathcal{V}_{\text{tgt}}$ obtained from Voxel FlowEdit (Sec. 3.2), we refine local geometry by updating the latent feature vectors $\{\mathbf{z}_{\mathbf{p}}\}_{\mathbf{p} \in \mathcal{V}_{\text{tgt}}}$ within the editable region, while anchoring the unedited ones to the source distribution. Let $\mathcal{M}_z$ denote the per-latent edit mask derived from the mesh-space mask $\mathcal{M}$.

Note that $\mathbf{v}_{\theta}$ here operates on the local latent features $\mathbf{z}$. At each discrete step k , the latent update follows a repainting process:

$$\begin{aligned} \mathbf{z}_{k-1} &= \mathcal{M}_z \odot \left[\mathbf{z}_k + \Delta t \mathbf{v}_{\theta}(\mathbf{z}_k, t_k | I^{\text{tgt}}) \right] \\ &+ (1 - \mathcal{M}_z) \odot \left[(1 - t_{k-1}) \mathbf{z}^{\text{src}} + t_{k-1} \epsilon_{k-1} \right], \end{aligned} \quad (5)$$

where $\mathbf{z}^{\text{src}}$ is the initial source latent vector at t_0 , and $\epsilon_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ denotes Gaussian noise.

The two masked terms jointly ensure local refinement and global preservation: the first term applies target-conditioned velocity for structural refinements in the editable region, whereas the second term replays the forward-diffused source trajectory to maintain global appearance and geometry. In practice, a softly feathered mask $\mathcal{M}_z = \text{blur}(\mathcal{M}_z; \sigma_b)$ is used to prevent seam artifacts.

After reaching $k = 0$, the final edited latent field $\mathbf{Z} = (\mathcal{V}_{\text{tgt}}, \{\mathbf{z}_{\mathbf{p}}\}_{\mathbf{p} \in \mathcal{V}_{\text{tgt}}})$ is decoded by TRELIS to produce the refined 3D mesh.

3.4. Texture Refinement

Given the edited 3D mesh decoded from the preceding stages, we optionally apply a texture refinement module to

enhance appearance realism.

Control Branch. The Control Branch serves to inject precise geometric guidance for the subsequent multi-view synthesis. It is constructed by combining a frozen ControlNet [55] with a trainable Ctrl-Adapter [25]. The ControlNet receives per-view normal maps $\{\mathbf{N}_v\}$ rendered from the edited geometry and extracts multi-scale spatial features that encode local surface cues. The Ctrl-Adapter then learns to align and inject these control features into the main generative network, effectively acting as a geometry-aware conditioning module. This design enables precise and efficient control over texture synthesis guided strictly by the edited 3D shape.

Generation Branch. Conditioned on the geometric guidance extracted by the Control Branch, we adopt the multiview-diffusion architecture from ERA3D [22] to synthesize multi-view images consistent with the guiding geometry. The generation backbone takes the edited image I^{tgt} as primary context, using the control features $\mathbf{C}$ to ensure the output is consistent with the edited geometry. This network synthesizes six geometry-consistent auxiliary views $\{I'_v\}_{v=1}^6$ under predefined camera poses. This synthesis process propagates fine appearance details onto the novel viewpoints, providing reliable texture information for the following fusion stage. During training, only the parameters of the Ctrl-Adapter are updated to learn the feature alignment, while the ControlNet and the Era3D backbone remain frozen.

Texture Fusion. The final generated appearance is transferred back to the UV space via a robust fusion process. The synthesized views $\{I'_v\}$ are projected onto the 3D mesh and fused into the final UV texture $\mathbf{T}$ using a visibility-aware, mask-weighted blending scheme. This process uses the softly feathered edit mask $\tilde{\mathcal{M}}$ to prioritize integration in the edited regions, while strictly retaining the original appearance in unedited areas. This efficient transfer yields significantly sharper and more realistic textures for the edited assets.

4. Experiments

4.1. Implementation Details

For Voxel FlowEdit, we adopt a target-side classifier-free guidance (CFG) scale between 5 and 15, while the source-side CFG is fixed to 5. The latent ODE is discretized by 25 sampling steps, and the edit velocity $\mathbf{v}_{\text{edit}}$ is computed by averaging over $n_{\text{avg}} \in \{2, 4\}$ to improve stability. For the regularization terms, we normalize the silhouette-guidance gradient so that its ℓ_2 norm matches that of $\mathbf{v}_{\text{edit}}$, allowing a weighting coefficient of 0.2 for the silhouette term. The

trajectory-consistency residual is weighted by 0.1.

The normal-guided Ctrl-Adapter is trained on a subset of Objaverse [8], where each asset is rendered into six views of size 512×512 with corresponding normal maps. The adapter is trained to synthesize auxiliary views that guide the texture refinement stage.

We construct our evaluation set using 100 3D assets collected from Sketchfab Website, the NPHM [10] dataset for real human heads, the THuman2.1 [54] dataset for clothed human bodies, and the Objaverse [8] dataset for objects.

4.2. Comparisons

Baselines. We compare our method against several representative works. (1) TRELLIS [48], which serves as our generative backbone. We feed the edited target image I^{tgt} directly into its 3D generation pipeline and evaluate its ability to reproduce the edit while preserving consistency with the original 3D model. (2) MVEdit [6], a multi-view diffusion framework that uses a training-free 3D adapter to jointly denoise rendered views and output textured meshes. (3) Vox-E [40], a voxel-based volumetric editing method that learns a volumetric representation from oriented images and edits existing 3D objects under diffusion priors. (4) Instant3dit [2], a feed-forward multiview inpainting framework for fast 3D editing via view generation and reconstruction. (5) VoxHammer [21], a feed-forward 3D editing framework built upon TRELLIS that uses an image and a mask to guide the editing process. Since methods (2)-(4) are primarily text-guided frameworks rather than direct image-driven 3D generation, we provide them with a unified text prompt that semantically describes our image-based edit to ensure a fair comparison.

Qualitative Comparison. As shown in Fig. 4, our method produces clean, view-consistent edits while preserving the source asset’s global structure. Compared to TRELLIS and VoxHammer, our framework avoids overfitting and better maintains identity consistency with the original 3D model. MVEdit generates plausible textures but introduces negligible geometric changes and view-wise inconsistency. Both Vox-E and Instant3dit fail to maintain structural integrity under complex edits, with Vox-E also requiring significantly longer inference time. Overall, our approach achieves a superior balance of edit controllability, identity preservation, and visual fidelity.

Quantitative Comparison. We evaluate performance using four widely adopted quantitative metrics: CLIP-T (text-image alignment) [37], DINO-I (perceptual quality) [5], LPIPS (perceptual similarity) [56], and FID (distributional realism) [14]. As shown in Tab. 1, our method consistently achieves the best performance across all four metrics. These results confirm that our framework produces higher overall



Figure 4. Qualitative comparison. Our method achieves clean geometry and consistent appearance across multiple views, faithfully realizing the target edits while strictly preserving unedited regions. Competing methods either retain the original geometry (MVEdit) or exhibit strong structural distortion and inconsistency (Vox-E, Instant3dit). Although VoxHammer also utilizes mask inputs to guide the editing process, it struggles to maintain high fidelity outside the masked areas.

Table 1. Quantitative comparison on the 3D editing benchmark. Higher is better for CLIP-T, and DINO-I; lower is better for LPIPS and FID. Our method consistently achieves the best performance across all metrics.

Method	CLIP-T $\uparrow$	DINO-I $\uparrow$	LPIPS $\downarrow$	FID $\downarrow$
TRELLIS [48]	0.323	0.895	0.243	30.2
MVEdit [6]	0.267	0.851	0.282	67.6
Vox-E [40]	0.266	0.734	0.673	90.3
Instant3DiT [2]	0.285	0.874	0.286	49.7
VoxHammer [21]	0.295	0.938	0.193	31.7
Ours	0.326	0.952	0.138	25.8

3D quality and text alignment than representative baselines, yielding more coherent, high-fidelity 3D assets.

User Study. We also conduct a user study to further quan-

Table 2. User study results. Percentage of times each question is rated best (higher is better).

Question	Q1 $\uparrow$	Q2 $\uparrow$	Q3 $\uparrow$	Q4 $\uparrow$	Q5 $\uparrow$
Ours	88.98	94.63	94.92	97.51	97.00

titatively validate our method. Participants were asked to view the editing prompt, alongside videos rendered from both the source 3D assets and the 3D assets edited by our method and four competing methods, and then respond to a series of questions:

- **Q1:** Which method best follows the given input prompt? (*Prompt Preservation*)
- **Q2:** Which method best retains the geometry and texture of the unedited regions? (*Identity Preservation*)
- **Q3:** Which method best produces edited geometry and



Figure 5. Comparison between without and with Flow Guidance. Disabling $\mathbf{G}_{\text{sil}}$ and ξ_{traj} jointly leads to structural collapse and view-inconsistent deformation, whereas enabling both yields stable and silhouette-aligned geometry.

texture? (3D Editing Quality)

- **Q4:** Which method best maintains 3D consistency? (3D Consistency)
- **Q5:** Which method is best overall considering the above four aspects? (Overall)

We collected statistics from 46 participants across 10 groups of editing results. For a fair comparison, the video results for each case were randomly shuffled. As shown in Tab. 2, our method remarkably outperforms other methods in prompt preservation, identity preservation, 3D editing quality, and 3D consistency, and is rated as the best in overall quality. These results demonstrate that our method is highly favored by users, highlighting its effectiveness across various editing dimensions.

4.3. Ablation Studies

Guided Flow Regularization. We ablate the auxiliary guidance terms in Eq. (4) to assess their impact on structural stability. Specifically, we compare the base ODE update without auxiliary guidance against the full guided formulation with both the silhouette gradient and the trajectory correction enabled. We toggle these two terms jointly, since $\mathbf{G}_{\text{sil}}$ aligns the evolving structure with the target silhouette, while ξ_{traj} regularizes the dynamics by projecting $\mathbf{x}_t$ back onto the flow manifold. Using only one of them leads to unbalanced updates and unstable geometry. As shown in Fig. 5, removing both terms results in a combination of over-preservation and structural drift, whereas enabling both yields coherent and well-aligned deformations.

Texture Refinement. We further investigate the effect of the normal-guided appearance refinement module. This component restores high-frequency texture details and har-

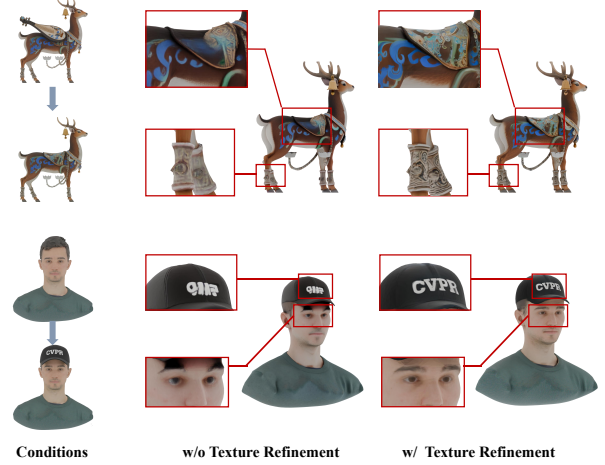


Figure 6. Comparison between without and with Texture Refinement. The refinement stage significantly enhances surface detail and view-consistent appearance.

monizes lighting across views via normal-conditioned feature modulation. Without Texture Refinement, the edited regions become noticeably blurrier and exhibit color bias, as illustrated in Fig. 6.

5. Conclusion

We have presented a unified feed-forward framework for 3D asset editing that integrates geometric transformation, latent-space refinement, and texture enhancement within a single generative pipeline. Our method builds upon the 3D-native TRELLIS representation, enabling coherent large-scale deformation and fine-grained texture editing directly from a single-view input. By combining Voxel FlowEdit, SLAT repainting, and normal-guided texture refinement, the proposed system effectively bridges 2D editing flexibility with 3D consistency and realism. Comprehensive experiments demonstrate that our approach achieves stable geometry, faithful texture preservation, and visually consistent results across diverse assets, offering a practical paradigm for efficient and controllable 3D editing.

Limitations. While our framework delivers robust and realistic edits, its performance remains bounded by the generative capacity of TRELLIS, particularly under extreme geometric modifications. Moreover, the normal-guided refinement currently operates on relatively low-resolution synthesized views, which would limit the recovery of very fine textures. We believe these limitations can be mitigated in future work through higher-resolution generation and stronger geometric priors.

References

- [1] Amir Barda, Vladimir Kim, Noam Aigerman, Amit Haim Bermano, and Thibault Groueix. Magicclay: Sculpting meshes with generative neural fields. In *SIGGRAPH Asia 2024 Conference Papers*, pages 1–10, 2024. 3
- [2] Amir Barda, Matheus Gadelha, Vladimir G Kim, Noam Aigerman, Amit H Bermano, and Thibault Groueix. Instant3dit: Multiview inpainting for fast editing of 3d objects. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 16273–16282, 2025. 2, 3, 6, 7, 1
- [3] Youcheng Cai, Runshi Li, and Ligang Liu. Mv2mv: Multiview image translation via view-consistent diffusion models. *ACM Transactions on Graphics (TOG)*, 43(6):1–12, 2024. 3
- [4] Mingdeng Cao, Xintao Wang, Zhongang Qi, Ying Shan, Xiaohu Qie, and Yinqiang Zheng. Masactrl: Tuning-free mutual self-attention control for consistent image synthesis and editing. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 22560–22570, 2023. 2, 3
- [5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021. 6
- [6] Hansheng Chen, Ruoxi Shi, Yulin Liu, Bokui Shen, Jiayuan Gu, Gordon Wetzstein, Hao Su, and Leonidas Guibas. Generic 3d diffusion adapter using controlled multi-view editing, 2024. 3, 6, 7, 1
- [7] Yongwei Chen, Yushi Lan, Shangchen Zhou, Tengfei Wang, and Xingang Pan. Sar3d: Autoregressive 3d object generation and understanding via multi-scale 3d vqvae. In *CVPR*, 2025. 2
- [8] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13142–13153, 2023. 6
- [9] Will Gao, Dilin Wang, Yuchen Fan, Aljaz Bozic, Tuur Stuyck, Zhengqin Li, Zhao Dong, Rakesh Ranjan, and Nikolaos Sarafianos. 3d mesh editing using masked lrms. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7154–7165, 2025. 3
- [10] Simon Giebenhain, Tobias Kirschstein, Markos Georgopoulos, Martin Rünz, Lourdes Agapito, and Matthias Nießner. Learning neural parametric head models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21003–21012, 2023. 6
- [11] Yuwei Guo, Ceyuan Yang, Anyi Rao, Maneesh Agrawala, Dahua Lin, and Bo Dai. Sparsectrl: Adding sparse controls to text-to-video diffusion models. In *European Conference on Computer Vision*, pages 330–348. Springer, 2024. 3
- [12] Ayaan Haque, Matthew Tancik, Alexei A Efros, Aleksander Holynski, and Angjoo Kanazawa. Instruct-nerf2nerf: Editing 3d scenes with instructions. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19683–19693. IEEE Computer Society, 2023. 2, 3
- [13] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross-attention control. In *ICLR*, 2023. 2, 3
- [14] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017. 6
- [15] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. Lrm: Large reconstruction model for single image to 3d. In *The Twelfth International Conference on Learning Representations*. 2
- [16] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022. 3
- [17] Bahjat Kawar, Shiran Zada, Oran Lang, Omer Tov, Huiwen Chang, Tali Dekel, Inbar Mosseri, and Michal Irani. Imagic: Text-based real image editing with diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6007–6017, 2023. 3
- [18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 2
- [19] Vladimir Kulikov, Matan Kleiner, Inbar Huberman-Spiegelglas, and Tomer Michaeli. Flowedit: Inversion-free text-based editing using pre-trained flow models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19721–19730, 2025. 3, 4, 5
- [20] Nupur Kumari, Bingliang Zhang, Richard Zhang, Eli Shechtman, and Jun-Yan Zhu. Multi-concept customization of text-to-image diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1931–1941, 2023. 3
- [21] Lin Li, Zehuan Huang, Haoran Feng, Gengxiong Zhuang, Rui Chen, Chunchao Guo, and Lu Sheng. Voxhammer: Training-free precise and coherent 3d editing in native 3d space. *arXiv preprint arXiv:2508.19247*, 2025. 3, 6, 7
- [22] Peng Li, Yuan Liu, Xiaoxiao Long, Feihu Zhang, Cheng Lin, Mengfei Li, Xingqun Qi, Shanghang Zhang, Wei Xue, Wenhan Luo, et al. Era3d: High-resolution multiview diffusion using efficient row-wise attention. *Advances in Neural Information Processing Systems*, 37:55975–56000, 2024. 6
- [23] Zhiqi Li, Yiming Chen, Lingzhe Zhao, and Peidong Liu. Controllable text-to-3d generation via surface-aligned gaussian splatting. In *2025 International Conference on 3D Vision (3DV)*, pages 1113–1123. IEEE, 2025. 2
- [24] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 300–309, 2023. 2
- [25] H Lin, J Cho, A Zala, and M Bansal. Ctrl-adapter: An efficient and versatile framework for adapting diverse controls to

- any diffusion model. *Proceedings of the International Conference on Learning Representations*, 2025. 6
- [26] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *11th International Conference on Learning Representations, ICLR 2023*, 2023. 3
- [27] Minghua Liu, Ruoxi Shi, Linghao Chen, Zhuoyang Zhang, Chao Xu, Xinyue Wei, Hansheng Chen, Chong Zeng, Jiayuan Gu, and Hao Su. One-2-3-45++: Fast single image to 3d objects with consistent multi-view generation and 3d diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10072–10083, 2024. 2
- [28] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9298–9309, 2023. 2
- [29] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. 3
- [30] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations*, 2021. 2, 3
- [31] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 2
- [32] Ron Mokady, Amir Hertz, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Null-text inversion for editing real images using guided diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6038–6047, 2023. 2, 3
- [33] Haomiao Ni, Changhao Shi, Kai Li, Sharon X Huang, and Martin Renqiang Min. Conditional image-to-video generation with latent flow diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18444–18455, 2023. 3
- [34] Michael Niemeyer, Lars Mescheder, Michael Oechsle, and Andreas Geiger. Differentiable volumetric rendering: Learning implicit 3d representations without 3d supervision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3504–3515, 2020. 2
- [35] Michael Oechsle, Songyou Peng, and Andreas Geiger. Unisurf: Unifying neural implicit surfaces and radiance fields for multi-view reconstruction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5589–5599, 2021. 2
- [36] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *ICLR*, 2023. 2
- [37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 6
- [38] Amit Raj, Srinivas Kaza, Ben Poole, Michael Niemeyer, Nataniel Ruiz, Ben Mildenhall, Shiran Zada, Kfir Aberman, Michael Rubinstein, Jonathan Barron, et al. Dreambooth3d: Subject-driven text-to-3d generation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2349–2359, 2023. 2
- [39] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 22500–22510, 2023. 3
- [40] Etai Sella, Gal Fiebelman, Peter Hedman, and Hadar Averbuch-Elor. Vox-e: Text-guided voxel editing of 3d objects. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 430–440, 2023. 3, 6, 7, 1
- [41] Yichun Shi, Peng Wang, Jianglong Ye, Long Mai, Kejie Li, and Xiao Yang. Mvdream: Multi-view diffusion for 3d generation. In *The Twelfth International Conference on Learning Representations*, 2024. 2
- [42] Jiaxiang Tang, Zhaoxi Chen, Xiaokang Chen, Tengfei Wang, Gang Zeng, and Ziwei Liu. Lgm: Large multi-view gaussian model for high-resolution 3d content creation. In *European Conference on Computer Vision*, pages 1–18. Springer, 2024. 2
- [43] Narek Tumanyan, Michal Geyer, Shai Bagon, and Tali Dekel. Plug-and-play diffusion features for text-driven image-to-image translation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1921–1930, 2023. 2, 3
- [44] Junjie Wang, Jiemin Fang, Xiaopeng Zhang, Lingxi Xie, and Qi Tian. Gaussianeditor: Editing 3d gaussians delicately with text instructions. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20902–20911. IEEE Computer Society, 2024. 2
- [45] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *Advances in neural information processing systems*, 36: 8406–8441, 2023. 2
- [46] Si-Tong Wei, Rui-Huan Wang, Chuan-Zhi Zhou, Baoquan Chen, and Wang Peng-Shuai. Octgpt: Octree-based multi-scale autoregressive models for 3d shape generation. In *SIGGRAPH*, 2025. 2
- [47] Jing Wu, Jia-Wang Bian, Xinghui Li, Guangrun Wang, Ian Reid, Philip Torr, and Victor Adrian Prisacariu. Gaussctrl: Multi-view consistent text-driven 3d gaussian splatting editing. In *European Conference on Computer Vision*, pages 55–71. Springer, 2024. 3
- [48] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation. In *Proceedings of the Computer Vision and Pattern*

Recognition Conference, pages 21469–21480, 2025. [2](#), [3](#), [6](#), [7](#)

- [49] Binxin Yang, Shuyang Gu, Bo Zhang, Ting Zhang, Xuejin Chen, Xiaoyan Sun, Dong Chen, and Fang Wen. Paint by example: Exemplar-based image editing with diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18381–18391, 2023. [3](#)
- [50] Xianghui Yang, Huiwen Shi, Bowen Zhang, Fan Yang, Jiacheng Wang, Hongxu Zhao, Xinhai Liu, Xinzhou Wang, Qingxiang Lin, Jiaao Yu, et al. Hunyuan3d 1.0: A unified framework for text-to-3d and image-to-3d generation. *arXiv preprint arXiv:2411.02293*, 2024. [2](#)
- [51] Jianglong Ye, Peng Wang, Kejie Li, Yichun Shi, and Heng Wang. Consistent-1-to-3: Consistent image to 3d view synthesis via geometry-aware diffusion models. In *2024 International Conference on 3D Vision (3DV)*, pages 664–674. IEEE Computer Society, 2024. [2](#)
- [52] Junliang Ye, Shenghao Xie, Ruowen Zhao, Zhengyi Wang, Hongyu Yan, Wenqiang Zu, Lei Ma, and Jun Zhu. Nano3d: A training-free approach for efficient 3d editing without masks, 2025. [3](#)
- [53] Quanzeng You, Hailin Jin, Zhaowen Wang, Chen Fang, and Jiebo Luo. Image captioning with semantic attention. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4651–4659, 2016. [3](#)
- [54] Tao Yu, Zerong Zheng, Kaiwen Guo, Pengpeng Liu, Qionghai Dai, and Yebin Liu. Function4d: Real-time human volumetric capture from very sparse consumer rgbd sensors. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR2021)*, 2021. [6](#)
- [55] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3836–3847, 2023. [3](#), [6](#)
- [56] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. [6](#)
- [57] Xiuming Zhang, Pratul P Srinivasan, Boyang Deng, Paul Debevec, William T Freeman, and Jonathan T Barron. Nerfactor: Neural factorization of shape and reflectance under an unknown illumination. *ACM Transactions on Graphics (ToG)*, 40(6):1–18, 2021. [2](#)
- [58] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, Yunfei Zhao, Haolin Liu, Shuhui Yang, Yifei Feng, Mingxin Yang, Sheng Zhang, Xianghui Yang, et al. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation. *CoRR*, 2025. [2](#)
- [59] Jingyu Zhuang, Chen Wang, Liang Lin, Lingjie Liu, and Guanbin Li. Dreameditor: Text-driven 3d scene editing with neural fields. In *SIGGRAPH Asia 2023 Conference Papers*, pages 1–10, 2023. [2](#), [3](#)

Easy3E: Feed-Forward 3D Asset Editing via Rectified Voxel Flow

Supplementary Material

A. Editing Efficiency

Table 3. Runtime comparison with different methods.

Method	Runtime
Vox-E [40]	37 min
MVEdit [6]	212 s
Instant3dit [2]	25 s
Ours	75 s

We compare the efficiency of our approach with three baselines: Instant3dit, MVEdit, and Vox-E, as shown in Tab. 3. Instant3dit is the fastest (25 seconds) due to its highly streamlined pipeline, but this compactness often limits its ability to handle complex disentanglement, resulting in lower fidelity. In contrast, MVEdit incurs a significantly higher computational cost of 212 seconds, as it relies on heavy iterative multi-view diffusion refinement. Vox-E is even more time-consuming, requiring full 3D optimization of the voxel grid, which takes approximately 37 minutes per edit. Our method operates in a sweet spot with a total runtime of 75 seconds. Adopting an efficient feed-forward design similar to Instant3dit, our approach eliminates the need for time-consuming per-scene optimization. However, unlike the unified pipeline of Instant3dit, we utilize a structured workflow decomposed into geometry editing (30 seconds), texture refinement (30 seconds), and back-projection (15 seconds). This 75 seconds duration allows us to achieve substantially better consistency and detail than Instant3dit, while remaining orders of magnitude faster than the optimization-heavy baselines.

B. Effectiveness of Texture Refinement

Fig. 7 presents the qualitative results obtained from our normal-guided Multi-view Diffusion Module. Conditioned on multi-view normal maps rendered from the input mesh and a single reference image, the module synthesizes a coherent sequence of multi-view images. As shown in the visualization, the generated images exhibit high-fidelity textures with intricate details. More importantly, benefiting from the structural guidance of surface normals, the results demonstrate rigorous cross-view consistency, where the object identity and geometric features remain stable across varying camera poses. This ensures that the subsequent texture back-projection step produces a seamless 3D model without alignment artifacts.

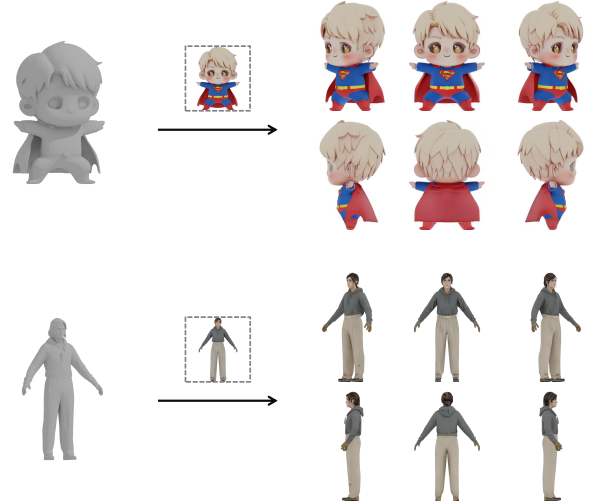


Figure 7. Visualizations of the normal-guided multi-view diffusion module. Taking rendered normal maps and a reference image as input, the module generates multi-view images that are both texture-rich and geometrically consistent, serving as robust priors for 3D texture recovery.

C. More Results

We present six supplementary examples in Fig. 8 to further validate the robustness of our approach. (a)-(c) demonstrate our capabilities on non-realistic objects; observe that the edited regions undergo significant geometric deformation, while the geometry of the unedited regions is strictly preserved. (d) illustrates the effectiveness of our method in scene-level editing. (e)-(f) showcase results on human subjects, where the clothing is successfully modified with high visual quality while maintaining the texture fidelity of the unedited body parts.

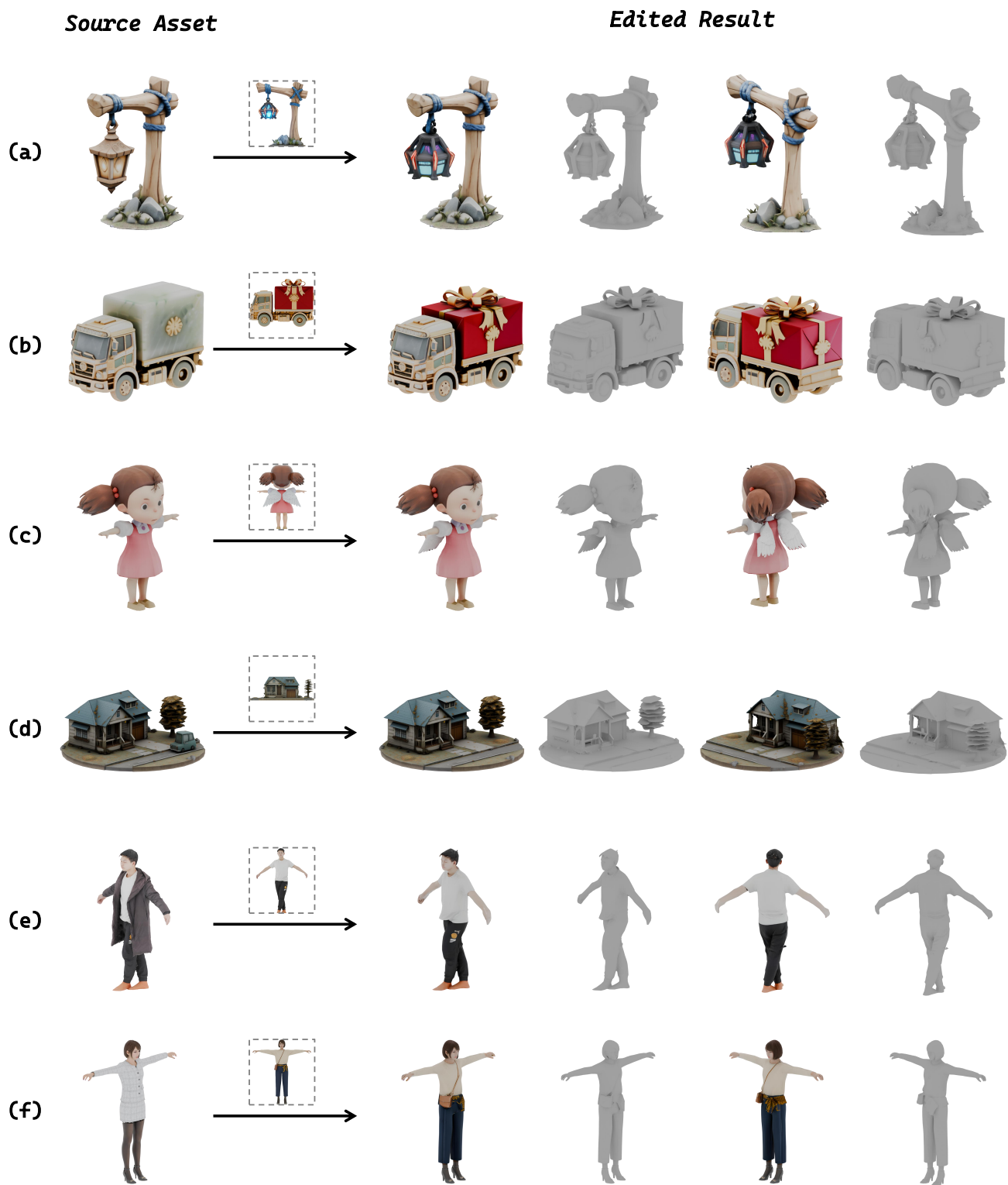


Figure 8. More visualization results.